Лабораторийн ажил №6

Сэдэв: Deadlock-ыг C хэл дээр симуляц хийх, шинжлэх, сайжруулах

Гүйцэтгэсэн: Н.Мөнхпүрэв, 23B1NUM0114

1. ОРШИЛ

Энэ лабораторийн ажилд Chapter 7: Deadlocks бүлгийн онолыг POSIX thread (pthread) ашигласан C програм дээр туршиж үзсэн. Хоёр thread хоёр mutex lock-ийг эсрэг дарааллаар авах үед програм гацдагийг ажиглаж, яагаад гацаж байгааг deadlock-ийн 4 нөхцөлтэй холбон тайлбарласан.

Дараа нь lock авах дарааллыг ижил болгож (lock ordering) deadlock-гүй хувилбар хийсэн. Нэмэлт даалгаврын хүрээнд pthread_mutex_trylock ашигласан хувилбар бичиж, хоёр аргыг харьцуулсан.

2. ЗОРИЛГО

Deadlock-ийн онолыг олон урсгалт програм дээр ажиглах, үүсэх шалтгааныг тодорхойлох, lock авах дарааллыг өөрчилж deadlock-оос сэргийлэх.

1. Жишээ кодын глобал хувьсагч, mutex, thread функц, main()-ийн үүргийг тайлбарлах.
2. Програмыг gcc -pthread-ээр хөрвүүлж ажиллуулаад мессежийн дарааллыг тэмдэглэх.
3. Програм хаана гацаж байгааг тодорхойлж, deadlock-ийн 4 нөхцөлийг код дээр тайлбарлах.
4. Хоёр thread хоёулаа lock1 → lock2 дарааллаар авах хувилбар боловсруулах.
5. Хоёр хувилбарын үр дүнг харьцуулж, нэмэлтээр trylock ашигласан өргөтгөл хийх.

3. ОНОЛЫН СУДАЛГАА

3.1 Lock, mutex, critical section

Mutex (mutual exclusion lock) нь нэг цагт зөвхөн нэг thread эзэмшиж болох түгжээ юм. pthread_mutex_lock() нь lock-ийг авна, хэрэв өөр thread барьж байвал суллагдтал хүлээнэ. pthread_mutex_unlock() нь lock-ийг суллана. Lock ба unlock хоёрын хоорондох, shared нөөцөд ханддаг кодын хэсгийг critical section гэнэ.

3.2 Deadlock ба түүний 4 нөхцөл

Deadlock гэдэг нь хоёр буюу түүнээс олон thread бие биенийхээ эзэмшиж буй нөөцийг хүлээсээр, хэн нь ч цааш ажиллаж чадахгүй болсон төлөв юм. Deadlock үүсэхийн тулд дараах 4 нөхцөл зэрэг биелэх ёстой. Аль нэгийг нь эвдвэл deadlock үүсэхгүй.

Нөхцөл	Тайлбар
Mutual Exclusion	Нөөцийг нэгэн зэрэг зөвхөн нэг thread эзэмшинэ.
Hold and Wait	Thread нэг lock-ийг барьсан атлаа өөр lock-ийг хүлээнэ.
No Preemption	Авсан lock-ийг бусад thread хүчээр булаан авч чадахгүй.
Circular Wait	Thread-үүд тойрог хүлээлт үүсгэнэ: T1 → T2 → ... → T1.

3.3 Deadlock-оос сэргийлэх аргууд

Lock ordering нь бүх lock-д дугаар өгч, thread бүр тэдгээрийг зөвхөн өсөх дарааллаар авах дүрэм юм. Энэ нь circular wait нөхцөлийг эвддэг. Харин trylock нь lock завгүй бол хүлээхгүй шууд буцдаг тул thread барьсан lock-оо сулласнаар hold and wait нөхцөлийг эвддэг.

4. ХЭРЭГЖҮҮЛЭЛТ

Хувилбар бүрийг тусдаа эх файлд бичиж, жижиг Makefile ашигласан. Заавар дээрх жишээ кодыг deadlock.c нэртэй хадгалсан.

Файл	Үүрэг	Тайлбар
deadlock.c	Жишээ код	Thread-үүд lock-ийг эсрэг дарааллаар авна. Deadlock үүснэ.
safe.c	Сайжруулсан	Хоёр thread хоёулаа lock1 → lock2 дарааллаар авна.
trylock.c	Нэмэлт даалгавар	Эсрэг дараалал, гэхдээ 2 дахь lock-ийг trylock-оор авна.
Makefile	Хөрвүүлэх	make командаар 3 програмыг -pthread-тэй хөрвүүлнэ.

4.1 Жишээ код: deadlock.c

Хэсэг	Үүрэг
pthread_mutex_t lock1, lock2	Глобал хувьсагч. Хоёр thread хоёулаа хандах ёстой тул глобал. Хоёр нөөцийг төлөөлнө.
thread1_func()	lock1-ийг авч, 1 секунд унтаад, lock2-ийг авна. Дараа нь lock2, lock1 гэж суллана.
thread2_func()	lock2-ийг авч, 1 секунд унтаад, lock1-ийг авна. Дараалал нь thread1-ээс эсрэг.
sleep(1)	Эхний lock-оо авсны дараа хүлээнэ. Энэ хугацаанд нөгөө thread өөрийн эхний lock-ийг амжиж авна.
main()	Mutex-үүдийг init хийж, 2 thread үүсгэнэ (pthread_create), хоёулаа дуустал хүлээнэ (pthread_join), mutex-ийг destroy хийгээд "Main: програм дууслаа" гэж хэвлэнэ.

Deadlock үүсгэдэг гол хэсэг нь хоёр функцийн lock авах дараалал юм:

```
/* thread1_func */                  /* thread2_func */
pthread_mutex_lock(&lock1);         pthread_mutex_lock(&lock2);
sleep(1);                           sleep(1);
pthread_mutex_lock(&lock2);         pthread_mutex_lock(&lock1);
```

4.2 Сайжруулсан хувилбар: safe.c

Хоёр thread нэг safe_thread_func функцийг ажиллуулна, thread-ийн нэрийг pthread_create-ийн сүүлийн аргументаар дамжуулна. Хоёулаа lock1-ийг эхэлж, дараа нь lock2-ийг авна.

```
void* safe_thread_func(void* arg) {
    char* name = arg;
    pthread_mutex_lock(&lock1);
    sleep(1);
    pthread_mutex_lock(&lock2);
    printf("%s: critical section-д ажиллаж байна...\n", name);
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}
```

4.3 Нэмэлт даалгавар: trylock.c

Lock-ийн дарааллыг анхны кодтой адил эсрэг хэвээр үлдээсэн. Харин хоёр дахь lock-ийг pthread_mutex_trylock()-оор авна. Энэ функц lock завгүй бол хүлээлгүй шууд EBUSY буцаана. Тэгвэл thread эхний lock-оо суллаад, хэсэг хүлээж (Thread 1 – 0.2 сек, Thread 2 – 0.7 сек) дахин оролдоно. Хүлээх хугацааг өөр өөр болгосон нь хоёр thread байнга зэрэг оролдохоос (livelock) сэргийлнэ.

```
if (pthread_mutex_trylock(&lock2) == 0) {
    printf("Thread 1: lock2 авлаа\n");
    break;
}
printf("Thread 1: lock2 завгүй, lock1-ийг сулласан\n");
pthread_mutex_unlock(&lock1);
usleep(200000);
```

5. ТУРШИЛТ БА ҮР ДҮН

Орчин: Ubuntu, Linux kernel 7.0.0-38-generic, gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0, POSIX Threads. Хөрвүүлэх команд:

```
gcc deadlock.c -o deadlock -pthread
make
```

5.1 deadlock.c (3 удаа)

№	Эхэлж хэвлэсэн	Сүүлийн мөр	Үр дүн
1	Thread 2	Thread 1: lock2 авах гэж байна...	Гацсан
2	Thread 1	Thread 2: lock1 авах гэж байна...	Гацсан
3	Thread 1	Thread 2: lock1 авах гэж байна...	Гацсан

Програмын гаралт (ажиллуулалт 2, Ctrl+C дарж зогсоосон):

```
Thread 1: lock1 авах гэж байна...
Thread 1: lock1 авлаа
Thread 2: lock2 авах гэж байна...
Thread 2: lock2 авлаа
Thread 1: lock2 авах гэж байна...
Thread 2: lock1 авах гэж байна...
^C
```

3 удаа бүгд 6 мөр хэвлээд зогссон. "lock2 авлаа", "ажил дууслаа", "Main: програм дууслаа" гэсэн мөрүүд хэзээ ч гарсангүй. Аль thread түрүүлж эхлэх нь scheduler-ээс хамаарч өөр байсан ч эцсийн үр дүн нь ижил.

Програм хаана гацаж байна вэ?
Thread 1 нь pthread_mutex_lock(&lock2) мөр дээр, Thread 2 нь pthread_mutex_lock(&lock1) мөр дээр зогссон. Main thread нь pthread_join(t1, NULL) дээр хүлээсээр байна.

Deadlock яагаад үүсэв?
Thread 1 lock1-ийг барьж, lock2-ийг хүлээж байна. lock2-ийг Thread 2 барьж байгаа бөгөөд тэр нь lock1-ийг хүлээж байна. Хоёулаа нөгөөгөө суллахыг хүлээж байгаа ч аль нь ч суллахгүй тул хүлээлт хэзээ ч дуусахгүй.

```
Thread 1 --барьж байна--> lock1 --хүлээж байна--> Thread 2
Thread 2 --барьж байна--> lock2 --хүлээж байна--> Thread 1
```

5.2 Deadlock-ийн 4 нөхцөл энэ код дээр

Нөхцөл	Код дээрх жишээ
Mutual Exclusion	pthread_mutex_t нэг цагт нэг л thread-д байна. lock1-ийг Thread 1 барьж байхад Thread 2 авч чадахгүй.
Hold and Wait	Thread 1 lock1-ээ суллалгүй барьсаар pthread_mutex_lock(&lock2) дээр хүлээнэ. Thread 2 ч мөн адил lock2-оо барьсаар байна.
No Preemption	Mutex-ийг зөвхөн эзэмшигч thread нь unlock хийж суллана. OS эсвэл нөгөө thread хүчээр авч чадахгүй.
Circular Wait	Thread 1 → lock2 (Thread 2-т) → Thread 2 → lock1 (Thread 1-д) → Thread 1. Тойрог үүссэн.

5.3 safe.c

Програмын гаралт (2 удаа ажиллуулсан, 2 удаа ижил, ойролцоогоор 2.0 секунд):

```
Thread 1: lock1 авах гэж байна...
Thread 1: lock1 авлаа
Thread 2: lock1 авах гэж байна...
Thread 1: lock2 авах гэж байна...
Thread 1: lock2 авлаа
Thread 1: critical section-д ажиллаж байна...
Thread 1: ажил дууслаа
Thread 2: lock1 авлаа
Thread 2: lock2 авах гэж байна...
Thread 2: lock2 авлаа
Thread 2: critical section-д ажиллаж байна...
Thread 2: ажил дууслаа
Main: deadlock үүсгэлгүй програм дууслаа
```

Thread 2 lock1 дээр хүлээж байх үедээ ямар ч lock бариагүй байна. Тиймээс Thread 1 lock2-ийг саадгүй аваад, ажлаа дуусгаж хоёр lock-оо суллана. Дараа нь Thread 2 lock1, lock2-ийг авч үргэлжилнэ.

5.4 trylock.c (3 удаа)

№	Thread 1 оролдлого	Thread 2 оролдлого	Дууссан эсэх
1	4	4	Тийм
2	1	2	Тийм
3	2	1	Тийм

Програмын гаралт (ажиллуулалт 2):

```
Thread 2: lock2 авлаа (оролдлого 1)
Thread 1: lock1 авлаа (оролдлого 1)
Thread 2: lock1 завгүй, lock2-ийг сулласан
Thread 1: lock2 авлаа
Thread 1: critical section-д ажиллаж байна...
Thread 1: ажил дууслаа (1 оролдлого)
Thread 2: lock2 авлаа (оролдлого 2)
Thread 2: lock1 авлаа
Thread 2: critical section-д ажиллаж байна...
Thread 2: ажил дууслаа (2 оролдлого)
Main: trylock ашиглаж deadlock-ооc гарлаа
```

Ажиллуулалт 1-д хоёр thread хэд хэдэн удаа зэрэг lock-оо суллаж, дахин зэрэг авсан (4 оролдлого). Энэ нь livelock-ийн шинж: thread-үүд гацаагүй, ажиллаж байгаа ч урагшлахгүй. Хүлээх хугацаа өөр өөр тул тэд эцэст нь давхцахаа больж дууссан.

5.5 Харьцуулсан шинжилгээ

Үзүүлэлт	deadlock.c	safe.c	trylock.c
Lock авах дараалал	T1: lock1 → lock2, T2: lock2 → lock1	Хоёулаа lock1 → lock2	Эсрэг, 2 дахь нь trylock
Үр дүн	3/3 удаа гацсан	2/2 удаа дууссан	3/3 удаа дууссан
"Main: ... дууслаа"	Хэвлэгдээгүй	Хэвлэгдсэн	Хэвлэгдсэн
Эвдэрсэн нөхцөл	Байхгүй (4 нөхцөл бүгд биелсэн)	Circular Wait	Hold and Wait
Хугацаа	Хязгааргүй (Ctrl+C)	~2 секунд	~2.2–6.1 секунд
Сул тал	–	Thread 2 нь Thread 1-ийг хүлээнэ	Livelock гарч болно, код төвөгтэй

Deadlock яагаад арилсан бэ?

Бүх thread lock-уудыг нэг ижил дарааллаар авбал lock2-ийг барьж байгаа thread заавал lock1-ийг аль хэдийн барьсан байна. Тэгэхээр "lock2-ийг барьчихаад lock1-ийг хүлээх" тохиолдол гарахгүй, тойрог хүлээлт үүсэх боломжгүй. Бусад 3 нөхцөл хэвээрээ ч circular wait алга болсон тул deadlock үүсэхгүй.

6. ТАЙЛАНГИЙН АСУУЛТУУД

1. Deadlock болон starvation хоёрын ялгаа юу вэ?

Deadlock-д бүлэг thread бие биенээ хүлээж, аль нь ч хэзээ ч урагшлахгүй. Starvation-д систем ажиллаж, бусад thread нөөц авч байгаа ч нэг thread-д ээлж байнга ирэхгүй (жишээ нь priority бага). Deadlock гаднаас оролцохгүй бол арилахгүй, starvation нөхцөл өөрчлөгдвөл өөрөө арилж болно.

2. sleep(1)-ийг арилгавал deadlock үргэлж үүсэх үү?

Үгүй. sleep(1)-гүй хувилбарыг 200 удаа ажиллуулахад (1 секундын timeout) 191 удаа дууссан, 9 удаа гацсан. Нэг thread хоёр lock-оо маш хурдан аваад суллачихдаг тул нөгөө нь ихэнхдээ амжихгүй. Гэхдээ scheduler thread-ийг яг эхний lock-ийн дараа солих үед deadlock гарна. sleep нь deadlock-ийг үүсгэдэггүй, зөвхөн гарах магадлалыг бараг 100% болгодог. Алдаа код дотор байсаар байна.

3. trylock ашиглах нь deadlock-оос бүрэн сэргийлж чадах уу?

Зөв ашиглавал deadlock-оос сэргийлнэ, учир нь thread lock барьсаар хүлээхгүй. Гэхдээ бүх асуудлыг шийдэхгүй: livelock үүсч болно (миний туршилтад 4 удаа давтагдсан), нэг thread байнга ялагдаж starvation болж болно, мөн бүтэлгүйтэх бүрт барьсан бүх lock-оо суллахаа нэг газар мартвал deadlock буцаж гарна.

4. 3 thread, 3 lock бүхий илүү төвөгтэй deadlock-ийг хэрхэн дүрслэх вэ?

Resource allocation graph ашиглана. T1 нь L1-ийг барьж L2-ийг хүлээнэ, T2 нь L2-ийг барьж L3-ийг хүлээнэ, T3 нь L3-ийг барьж L1-ийг хүлээнэ:

```
T1 -> L2 -> T2 -> L3 -> T3 -> L1 -> T1
```

Граф дээр тойрог байгаа бөгөөд lock бүр нэг instance-тай тул энэ нь deadlock. Шийдэл нь мөн адил: бүгд L1 → L2 → L3 дарааллаар авах.

5. Бодит үйлдлийн системд ямар төрлийн нөөцүүд дээр deadlock түгээмэл тохиолдож болох вэ?

Kernel доторх mutex, spinlock; өгөгдлийн сангийн мөр болон хүснэгтийн lock (хоёр transaction эсрэг дарааллаар update хийх); файлын lock (flock); принтер зэрэг төхөөрөмж; pipe (хоёр процесс бие биенээсээ уншихыг хүлээх); санах ой.

7. ДҮГНЭЛТ

Энэ лабораторид хоёр thread хоёр lock-ийг эсрэг дарааллаар авахад deadlock 3 удаа бүгд үүссэнийг харлаа: програм 6 мөр хэвлээд зогссон, "Main: програм дууслаа" гарсангүй. Кодоос deadlock-ийн 4 нөхцөл бүгд олдсон. Lock-ийг хоёр thread-д ижил дарааллаар (lock1 → lock2) авахуулахад circular wait арилж, програм ~2 секундэд хэвийн дууссан. Нэмэлтээр trylock ашиглахад эсрэг дараалалтай ч програм дууссан, гэхдээ livelock гарч болохыг ажигласан. sleep(1)-гүй үед deadlock 200-аас 9 удаа л гарсан нь deadlock бол санамсаргүй, давтан гаргахад хэцүү алдаа гэдгийг харуулж байна. Иймд lock ordering бол хамгийн энгийн бөгөөд найдвартай арга юм.

8. ХАВСРАЛТ

ХАВСРАЛТ А. ЭХ КОДУУД

deadlock.c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1;
pthread_mutex_t lock2;

void* thread1_func(void* arg) {
    printf("Thread 1: lock1 авах гэж байна...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 1: lock1 авлаа\n");

    sleep(1);

    printf("Thread 1: lock2 авах гэж байна...\n");
    pthread_mutex_lock(&lock2);
    printf("Thread 1: lock2 авлаа\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    printf("Thread 1: ажил дууслаа\n");
    return NULL;
}

void* thread2_func(void* arg) {
    printf("Thread 2: lock2 авах гэж байна...\n");
    pthread_mutex_lock(&lock2);
    printf("Thread 2: lock2 авлаа\n");

    sleep(1);

    printf("Thread 2: lock1 авах гэж байна...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 2: lock1 авлаа\n");

    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);
    printf("Thread 2: ажил дууслаа\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock1, NULL);
    pthread_mutex_init(&lock2, NULL);

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock1);
    pthread_mutex_destroy(&lock2);
```

```
    printf("Main: програм дууслаа\n");
    return 0;
}
```

safe.c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1;
pthread_mutex_t lock2;

/* Хоёр thread хоёулаа lock1 -> lock2 дарааллаар авна */
void* safe_thread_func(void* arg) {
    char* name = arg;

    printf("%s: lock1 авах гэж байна...\n", name);
    pthread_mutex_lock(&lock1);
    printf("%s: lock1 авлаа\n", name);

    sleep(1);

    printf("%s: lock2 авах гэж байна...\n", name);
    pthread_mutex_lock(&lock2);
    printf("%s: lock2 авлаа\n", name);

    printf("%s: critical section-д ажиллаж байна...\n", name);

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    printf("%s: ажил дууслаа\n", name);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock1, NULL);
    pthread_mutex_init(&lock2, NULL);

    pthread_create(&t1, NULL, safe_thread_func, "Thread 1");
    pthread_create(&t2, NULL, safe_thread_func, "Thread 2");

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock1);
    pthread_mutex_destroy(&lock2);

    printf("Main: deadlock үүсгэлгүй програм дууслаа\n");
    return 0;
}
```

trylock.c

```
#include <stdio.h>
#include <pthread.h>
```

```
#include <unistd.h>

pthread_mutex_t lock1;
pthread_mutex_t lock2;

/*
 * Өргөтгөл: lock-ийн дараалал анхны кодтой адил эсрэг хэвээр.
 * Харин 2 дахь lock-ийг pthread_mutex_trylock-оор авна.
 * Авч чадахгүй бол эхний lock-оо суллаад, хэсэг хүлээгээд дахин оролдоно.
 */
void* thread1_func(void* arg) {
    int tries = 0;

    while (1) {
        tries++;
        pthread_mutex_lock(&lock1);
        printf("Thread 1: lock1 авлаа (оролдлого %d)\n", tries);
        sleep(1);

        if (pthread_mutex_trylock(&lock2) == 0) {
            printf("Thread 1: lock2 авлаа\n");
            break;
        }
        printf("Thread 1: lock2 завгүй, lock1-ийг сулласан\n");
        pthread_mutex_unlock(&lock1);
        usleep(200000);   /* 0.2 секунд */
    }

    printf("Thread 1: critical section-д ажиллаж байна...\n");
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    printf("Thread 1: ажил дууслаа (%d оролдлого)\n", tries);
    return NULL;
}

void* thread2_func(void* arg) {
    int tries = 0;

    while (1) {
        tries++;
        pthread_mutex_lock(&lock2);
        printf("Thread 2: lock2 авлаа (оролдлого %d)\n", tries);
        sleep(1);

        if (pthread_mutex_trylock(&lock1) == 0) {
            printf("Thread 2: lock1 авлаа\n");
            break;
        }
        printf("Thread 2: lock1 завгүй, lock2-ийг сулласан\n");
        pthread_mutex_unlock(&lock2);
        usleep(700000);   /* өөр хугацаа хүлээнэ, эс тэгвэл livelock болж болно */
    }

    printf("Thread 2: critical section-д ажиллаж байна...\n");
    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);
    printf("Thread 2: ажил дууслаа (%d оролдлого)\n", tries);
    return NULL;
}

int main() {
    pthread_t t1, t2;
```

```
    pthread_mutex_init(&lock1, NULL);
    pthread_mutex_init(&lock2, NULL);

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock1);
    pthread_mutex_destroy(&lock2);

    printf("Main: trylock ашиглаж deadlock-ooc гарлаа\n");
    return 0;
}
```

Makefile

```
CC = gcc
CFLAGS = -Wall -pthread
TARGETS = deadlock safe trylock

all: $(TARGETS)

deadlock: deadlock.c
    $(CC) $(CFLAGS) deadlock.c -o deadlock

safe: safe.c
    $(CC) $(CFLAGS) safe.c -o safe

trylock: trylock.c
    $(CC) $(CFLAGS) trylock.c -o trylock

clean:
    rm -f $(TARGETS)
```

ХАВСРАЛТ Б. ПРОГРАМЫН ГАРАЛТ

```
➜  lab6 git:(main) ✗ make
gcc -Wall -pthread deadlock.c -o deadlock
gcc -Wall -pthread safe.c -o safe
gcc -Wall -pthread trylock.c -o trylock
➜  lab6 git:(main) ✗ ./deadlock
Thread 1: lock1 авах гэж байна...
Thread 1: lock1 авлаа
Thread 2: lock2 авах гэж байна...
Thread 2: lock2 авлаа
Thread 1: lock2 авах гэж байна...
Thread 2: lock1 авах гэж байна...
^C
➜  lab6 git:(main) ✗ ./safe
Thread 1: lock1 авах гэж байна...
Thread 1: lock1 авлаа
Thread 2: lock1 авах гэж байна...
Thread 1: lock2 авах гэж байна...
Thread 1: lock2 авлаа
Thread 1: critical section-д ажиллаж байна...
Thread 1: ажил дууслаа
Thread 2: lock1 авлаа
Thread 2: lock2 авах гэж байна...
Thread 2: lock2 авлаа
Thread 2: critical section-д ажиллаж байна...
Thread 2: ажил дууслаа
Main: deadlock үүсгэлгүй програм дууслаа
➜  lab6 git:(main) ✗ ./trylock
Thread 1: lock1 авлаа (оролдлого 1)
Thread 2: lock2 авлаа (оролдлого 1)
Thread 1: lock2 завгүй, lock1-ийг сулласан
Thread 2: lock1 авлаа
Thread 2: critical section-д ажиллаж байна...
Thread 2: ажил дууслаа (1 оролдлого)
Thread 1: lock1 авлаа (оролдлого 2)
Thread 1: lock2 авлаа
Thread 1: critical section-д ажиллаж байна...
Thread 1: ажил дууслаа (2 оролдлого)
Main: trylock ашиглаж deadlock-оос гарлаа
➜  lab6 git:(main) ✗
```

Зураг 1. deadlock, safe, trylock програмуудын гаралт

deadlock (ажиллуулалт 1):

```
Thread 2: lock2 авах гэж байна...
Thread 2: lock2 авлаа
Thread 1: lock1 авах гэж байна...
Thread 1: lock1 авлаа
Thread 2: lock1 авах гэж байна...
Thread 1: lock2 авах гэж байна...
^C
```

trylock (ажиллуулалт 1, livelock-ийн жишээ):

```
Thread 2: lock2 авлаа (оролдлого 1)
Thread 1: lock1 авлаа (оролдлого 1)
Thread 2: lock1 завгүй, lock2-ийг сулласан
Thread 1: lock2 завгүй, lock1-ийг сулласан
Thread 1: lock1 авлаа (оролдлого 2)
Thread 2: lock2 авлаа (оролдлого 2)
Thread 1: lock2 завгүй, lock1-ийг сулласан
Thread 1: lock1 авлаа (оролдлого 3)
Thread 2: lock1 завгүй, lock2-ийг сулласан
Thread 2: lock2 авлаа (оролдлого 3)
Thread 1: lock2 завгүй, lock1-ийг сулласан
Thread 1: lock1 авлаа (оролдлого 4)
Thread 2: lock1 завгүй, lock2-ийг сулласан
Thread 1: lock2 авлаа
Thread 1: critical section-д ажиллаж байна...
Thread 1: ажил дууслаа (4 оролдлого)
Thread 2: lock2 авлаа (оролдлого 4)
Thread 2: lock1 авлаа
Thread 2: critical section-д ажиллаж байна...
Thread 2: ажил дууслаа (4 оролдлого)
Main: trylock ашиглаж deadlock-оос гарлаа
```